I/O Organization

Shobhanjana Kalita,
Dept. of CSE, Tezpur University

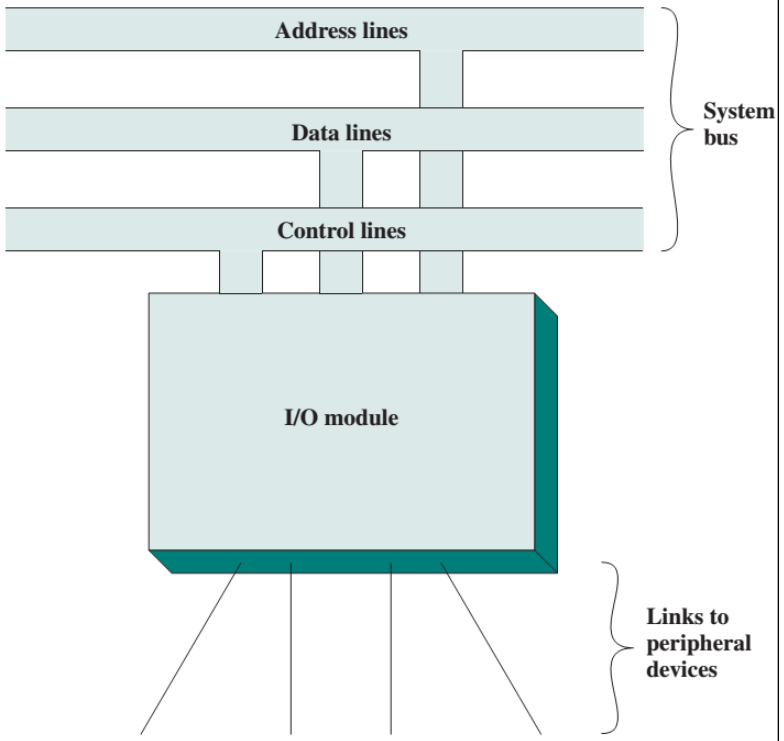
Slides courtesy: Computer Architecture and Organization, 9th Ed, W. Stallings

I/O Organization

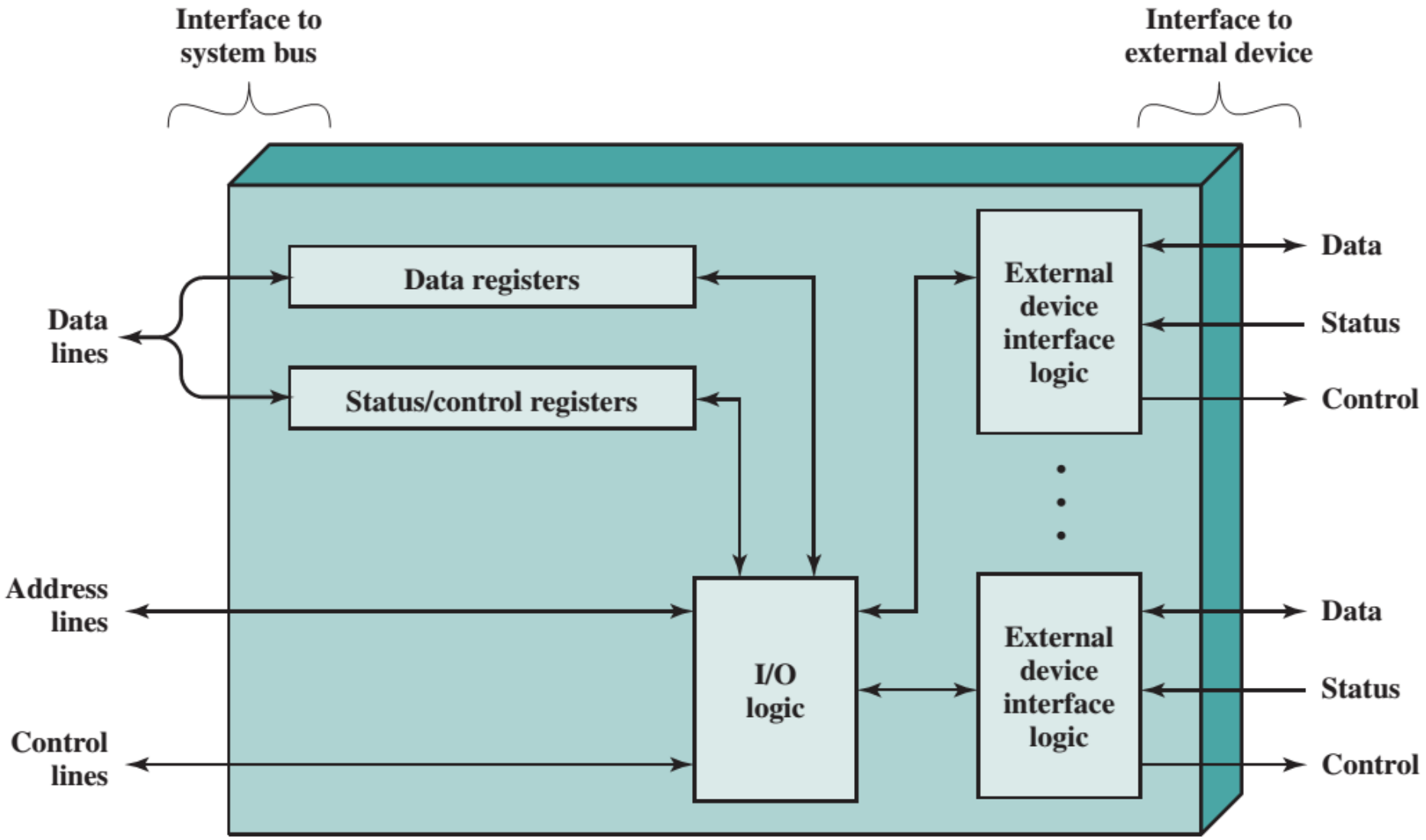
- I/O operations involve ***exchanging data*** between ***the external device*** and ***the computer***
- An **external device** attaches to the computer by a link to an **I/O module**
  - An external device connected to an I/O module is often referred to as a **peripheral device** or a **peripheral**
- **External devices** are of 3 categories:
  - **Human readable:** Suitable for communicating with the computer user; *e.g. display terminals, printer*
  - **Machine readable:** Suitable for communicating with equipment; *e.g. magnetic disks and tapes, sensors*
  - **Communication:** Suitable for communicating with remote devices; *e.g. exchanging with a remote device over the network*

I/O Module

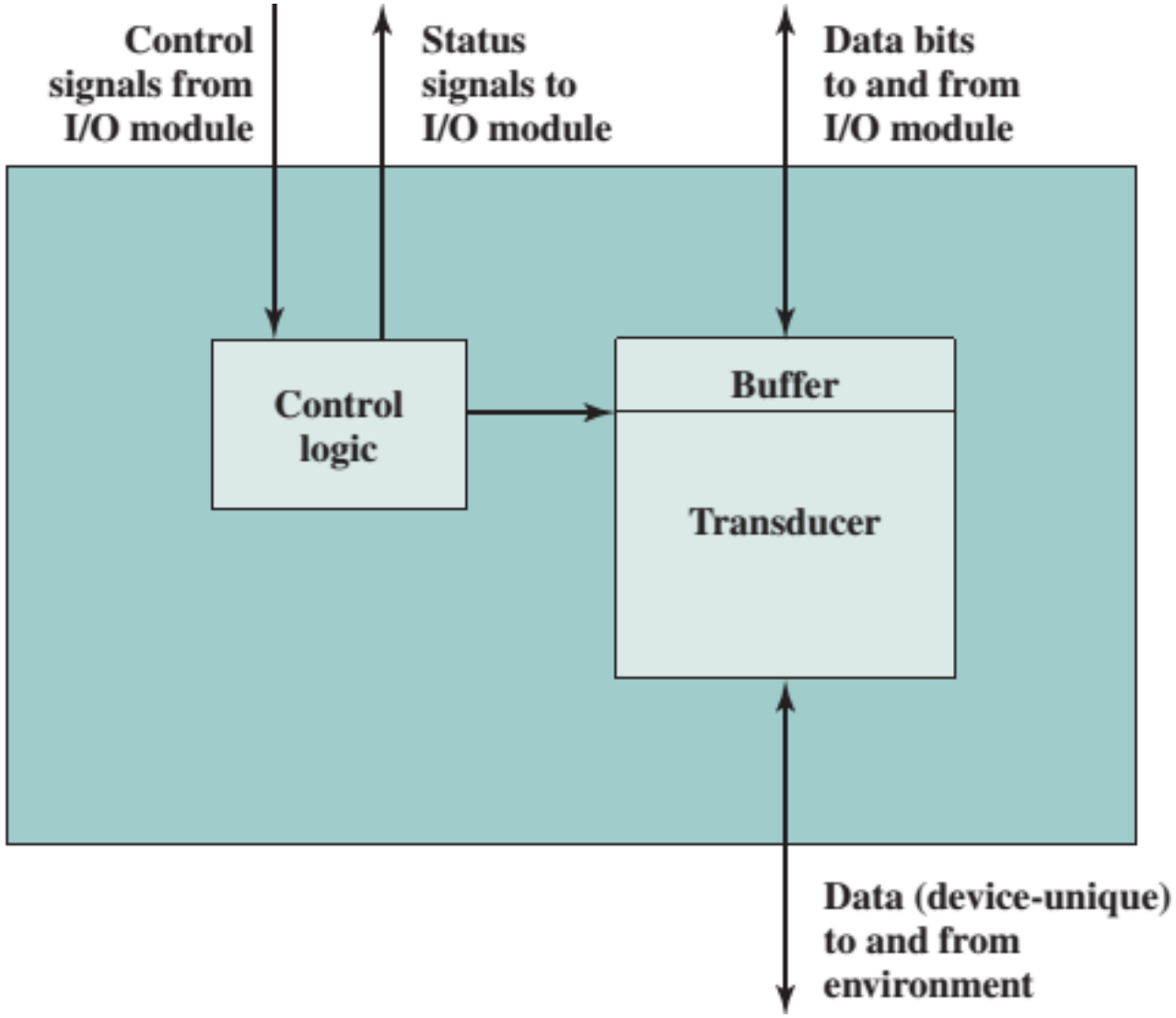
- The **interface between external device and I/O module** is in the form of control, data, and status signals
- **Control signals** determine the function that the device will perform
  - such as send data to the I/O module (OUTPUT or WRITE)
  - accept data from the I/O module (INPUT or READ)
  - report status, or perform some control function particular to the device (e.g., SEEK – position a disk head)
- **Data** are in the form of a set of bits to be sent to or received from the I/O module
- **Status signals** indicate the state of the device
  - E.g.: READY/NOT-READY to show whether the device is ready for data transfer



I/O Module


Interface to system bus
Interface to external device
Data registers
Status/control registers
Data lines
Address lines
Control lines
I/O logic
External device interface logic
External device interface logic
Data
Status
Control
Data
Status
Control

External Devices



- ***Control logic*** associated with the device controls the device's operation in response to direction from the I/O module
- ***Transducer*** converts data from electrical to other forms of energy during output and from other forms to electrical during input
- ***Buffer*** is associated with the transducer to temporarily hold data being transferred between the I/O module and the external environment; commonly 8 to 16 bits

External devices

- Keyboard/Monitor
  - Basic unit of exchange is the character
  - Associated with each character is a code, typically 7 or 8 bits in length – most commonly ASCII code
  - When the user presses a key – generates an electronic signal – transducer in keyboard interprets/translates into ASCII value –transmit to I/O module – store or send to display
  - To display – ASCII values are transmitted to external device from I/O module – transducer interprets/translates into electronic signals to the output device
- Disk Drive
  - Transducer can convert from bit sequence to magnetic patterns and vice versa

I/O Module – Function

- Functions or requirements of an I/O module
  - Control and timing
  - Processor communication
  - Device communication
  - Data buffering
  - Error detection

I/O Module – Function

- **Control and timing:**
  - Processor may communicate with many external devices at a time, in unpredictable patterns
  - Internal resources (main memory, system bus etc) are shared for all such communications
  - *Control and timing* are required, to coordinate the flow of traffic between internal resources and external devices
- Control of data transfer involves
  - Processor contacts I/O module to check status of the external device
  - I/O module returns the device status
  - If device status is READY, the processor requests the transfer of data to the I/O module
  - I/O module obtains a unit of data from the external device
  - Data are transferred from the I/O module to the processor.

I/O Module – Function

- **Processor communication**
  - I/O module communicates with processor for various purposes
  - **Command decoding:** I/O module accepts commands – READ SECTOR, WRITE SECTOR, SEEK track number etc. – from processor (sent as signals on the control bus)
  - **Data:** Data are exchanged between processor and I/O module over the data bus
  - **Status reporting:** Processor may need to know whether the I/O module is ready for data transfer – the status of the device
  - **Address recognition:** Each I/O device has an address and the I/O module should able to recognize a unique address for each peripheral device it controls

I/O Module – Functions

- **Device Communication**
  - I/O module must be able to communicate with the device to send instructions, obtain status information and obtain/send data
- **Data Buffering**
  - The transfer rate for main memory and processor is quite ***high*** and quite low for peripheral devices
  - Data coming from main memory are sent to an I/O module in a rapid burst – data are buffered in the I/O module and then sent to the peripheral device at its data rate
  - In opposite direction, data is buffered to optimally utilize main memory fast transfer rates – data is collected in a buffer and when enough data has been collected, it is sent in a rapid burst

I/O Module – Functions

- **Error detection**
  - I/O module should be able to detect and report errors to the processor
  - Errors could be – **(1)** mechanical and electrical malfunctions reported by the device (e.g., paper jam, bad disk track) or **(2)** unintentional changes to bit patterns during transmission from device to I/O module
  - Error detecting code, parity

I/O Operations

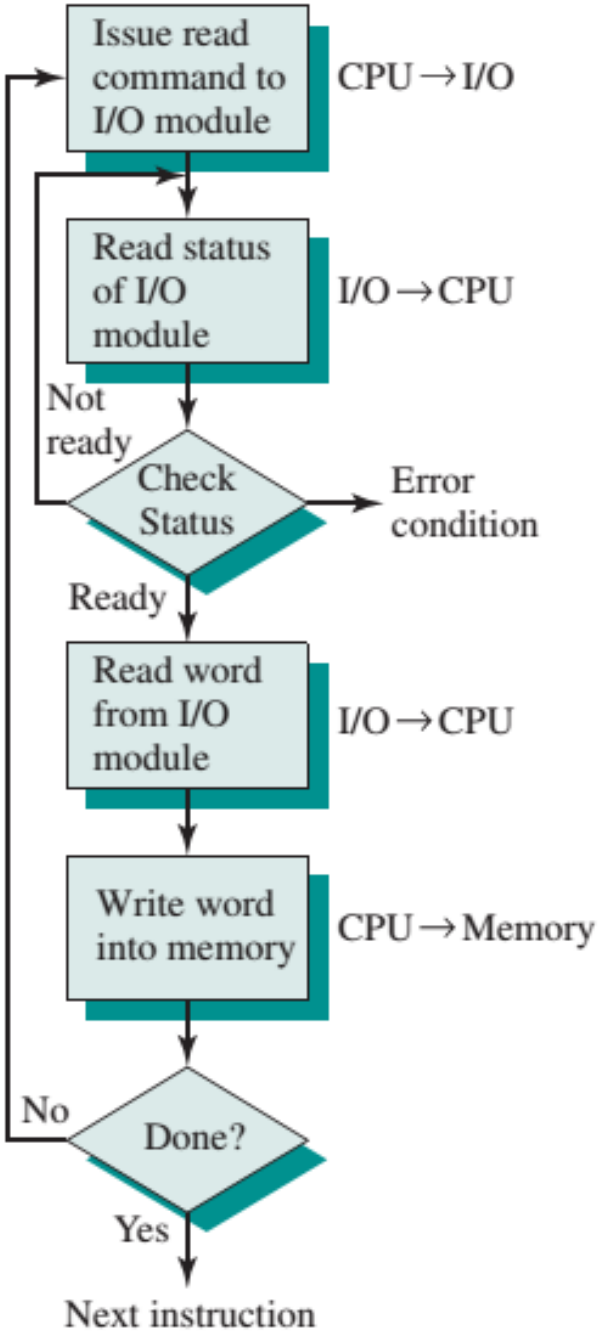
- 3 Different techniques for I/O operations
  - **Programmed I/O:** Data exchanged between processor and I/O module – executes a ***program*** that controls the entire I/O operation – sensing device status, sending a r/w command, and transferring data.
    - When the processor issues a command to the I/O module, it must wait until the I/O operation is complete. If the processor is faster than the I/O module, this is wasteful of processor time.
  - **Interrupt-driven I/O:** Processor issues an ***I/O command***, continues to execute other instructions, and is interrupted by the I/O module when work is done - processor is responsible for extracting/storing data from/to main memory
  - **Direct memory access (DMA):** I/O module and main memory exchange data directly, without processor involvement.

Programmed I/O

- Processor executes an I/O instruction by issuing a command to the appropriate I/O module
  - Processor issues the **address** (specifying the particular I/O module and external device) and the **I/O command**
- With programmed I/O, specified I/O module will perform the requested action and then set the appropriate bits in the I/O status register
  - Does not alert/inform the processor on completion
  - Responsibility of the processor **periodically to check the status** of the I/O module until it finds that the operation is complete

Programmed I/O

- Data are read in one word (e.g., 16 bits) at a time
- For each word that is read in, the processor must remain in a status-checking cycle until it determines that the word is available in the I/O module's data register
- Disadvantage of Programmed I/O:
  - a time-consuming process that keeps the processor busy needlessly



(a) Programmed I/O

Programmed I/O – Commands

- **Control:** Used to activate a peripheral and tell it what to do
  - For example, a magnetic-tape unit may be instructed to rewind or to move forward one record
  - These commands are tailored to the particular type of peripheral device
- **Test:** Used to test various status conditions associated with an I/O module and its peripherals
  - Whether powered on and available for use, if the most recent I/O operation is completed and if any errors occurred.
- **Read:** Causes the I/O module to ***obtain*** an item of data from the peripheral and place it in an internal buffer
  - The processor can then obtain the data item by requesting that the I/O module place it on the data bus
- **Write:** Causes the I/O module to ***take*** a data item from the data bus and subsequently transmit that data item to the peripheral

Programmed I/O – Instructions

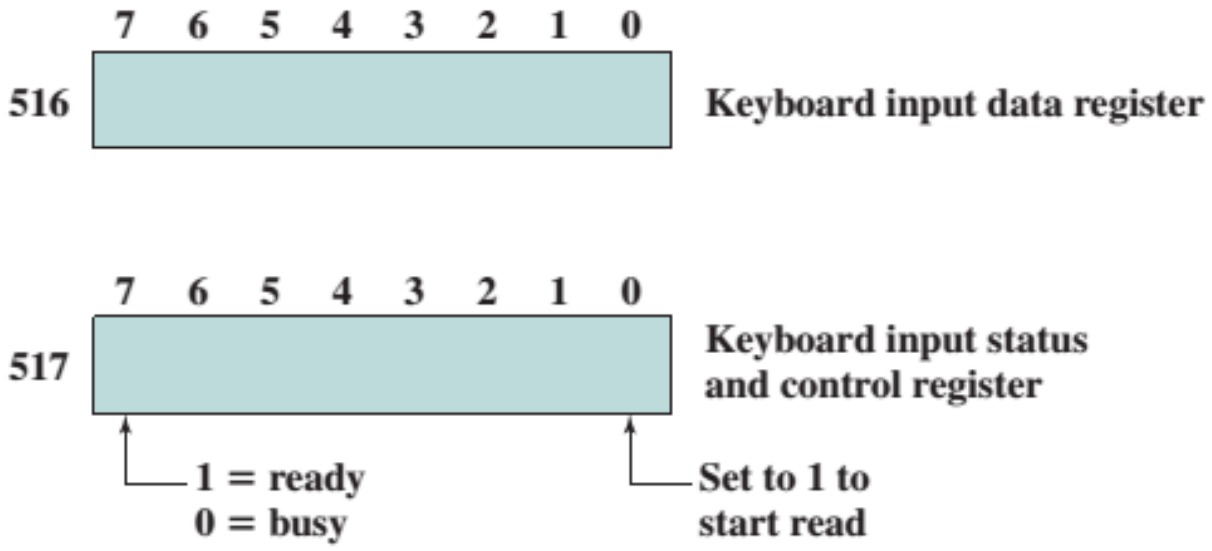
- I/O instructions (used in the program) are easily mapped into I/O commands (given by processor to the I/O module)
  - Often a simple one-to-one relationship
- Processor gives the I/O command with the address of the desired device
  - I/O module must interprets the address to determine if the command is for itself
- In shared bus systems, two addressing modes are possible
  - **Memory mapped I/O**
  - **Isolated I/O**

Programmed I/O

- **Memory mapped I/O**
  - A single address space for memory locations and I/O devices
  - The processor treats the **status** and **data registers of I/O** modules as **memory locations**
  - Uses same instructions to access both memory and I/O devices
  - **Same read and write lines on the bus** for both I/O and memory r/w operations
- **Isolated I/O**
  - **Separate read and write lines on the bus** for I/O and memory operations
  - Uses separate instructions to access memory and I/O devices – whether address is for an I/O device or memory location is obtained from the command
  - Full range of addresses may be available for both - **address space for I/O is isolated** from that for memory

Programmed I/O

- **Memory mapped I/O:**
  - 10-bit address
  - 512 MM locations(0–511)
  - 512 I/O addrs (512–1023)
  - **Disadv:** valuable memory address space is used up
- **Isolated I/O:**
  - I/O ports are accessible by special I/O commands which activate the I/O command lines on the bus
  - **Disadv:** Very few instructions



ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load AC	"1"	Load accumulator
	Store AC	517	Initiate keyboard read
202	Load AC	517	Get status byte
	Branch if Sign = 0	202	Loop until ready
	Load AC	516	Load data byte

(a) Memory-mapped I/O

ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load I/O	5	Initiate keyboard read
201	Test I/O	5	Check for completion
	Branch Not Ready	201	Loop until complete
	In	5	Load data byte

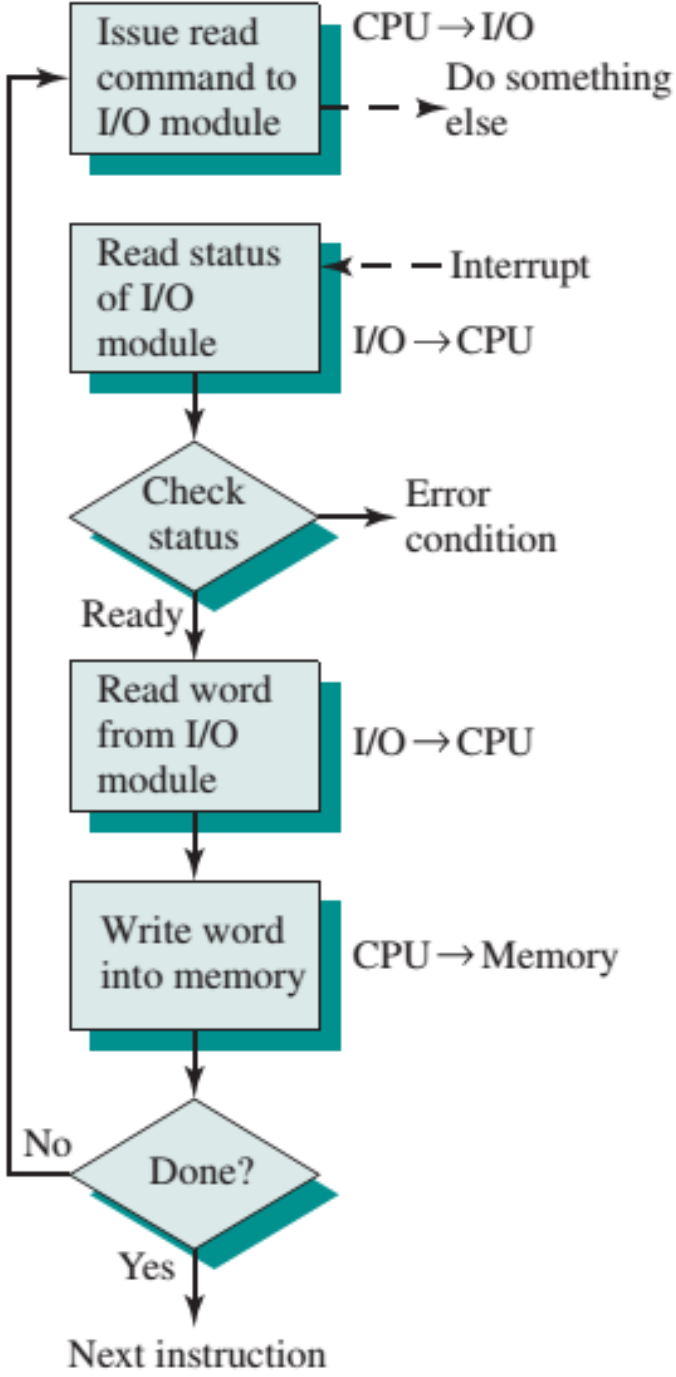
(b) Isolated I/O

Interrupt Driven I/O

- **Disadvantage of programmed I/O**
  - Processor has to wait a long time for the I/O module of concern to be ready
  - While waiting, processor must repeatedly interrogate the status of the I/O module
  - Level of the performance of the entire system degrades
- **Alternative:**
  - Processor issues an I/O command to a module – instead of waiting do some useful work
  - I/O module executes I/O command and informs processor on completion/when ready - **interrupt**
  - Processor on receiving interrupt, resumes the program execution

Interrupt Driven I/O

- Processor issues I/O command
- Saves the current context and goes off and does something else (e.g., some other program)
- At the end of each instruction cycle, the processor checks for interrupts
  - If I/O module has set interrupt, save context of the work it has been doing
  - Continue with the original program
- **Adv:** Interrupt I/O is more efficient than programmed I/O because it eliminates needless waiting



(b) Interrupt-Driven I/O

Interrupt Driven I/O

- **Design issues in implementing:**
  - In case of multiple I/O modules, how does the processor determine ***which device issued the interrupt*** **– device identification**
  - If multiple interrupts have occurred at the same time, how does the processor decide ***which one to process***

Interrupt Driven I/O

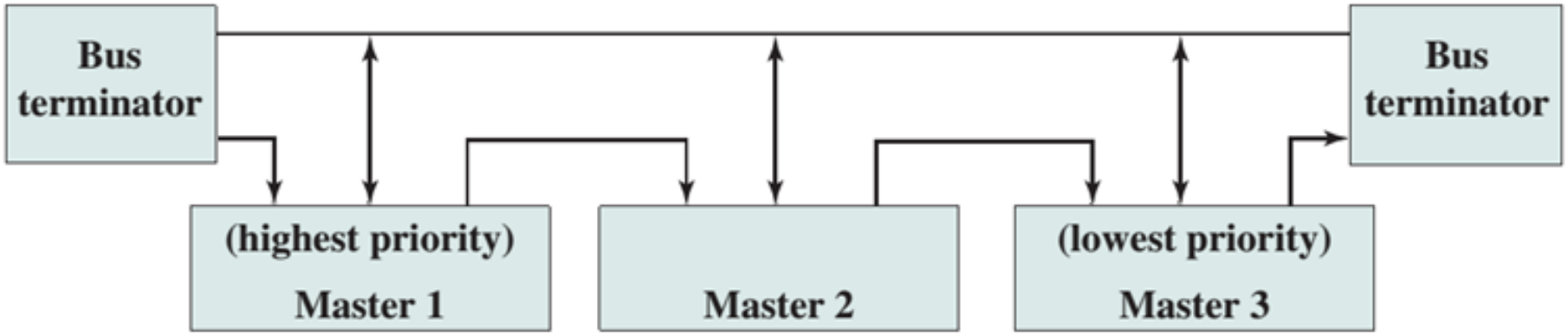
- **Device identification –** 4 different techniques:
  - Multiple interrupt lines
  - Software poll
  - Daisy chain (vectored, hardware polling)
  - Bus arbitration (vectored)
- **Multiple Interrupt Lines:**
  - Establish separate interrupt lines in the ***shared bus*** between the processor and the I/O modules for each external device
  - ***Not practical*** to allow more than a few lines between processor and I/O module – consequently, lines have to be shared between at least a few external devices

Interrupt Driven I/O

- **Software Polling:**
  - When processor detects an interrupt, it branches to an **interrupt-service routine**
  - ISR polls each I/O module device to determine which device caused the interrupt
  - Poll could be done using a separate command line (e.g. TESTI/O)
    - Processor raises TESTI/O
    - Places the address of a particular I/O module on the address lines
    - Module responds positively is it has set the interrupt
  - Or, each I/O module could contain an addressable status register
    - Processor reads the status register of each I/O module to identify the interrupting module
  - **Disadv:** Time consuming process

Interrupt Driven I/O

- **Daisy Chaining:**
  - More efficient – effectively ***hardware polling***
  - All I/O modules share a common **interrupt request line**
  - Processor senses interrupt and sends out an interrupt acknowledge - acknowledge line is daisy chained through the modules
  - Signal propagates through a series of I/O modules (daisy chained) until it gets to the requesting module
  - Requesting module typically responds by placing a word (called a **vector**) on the data lines – **vector** is either address or a unique ID
  - Avoids executing a general **ISR**



Interrupt Driven I/O

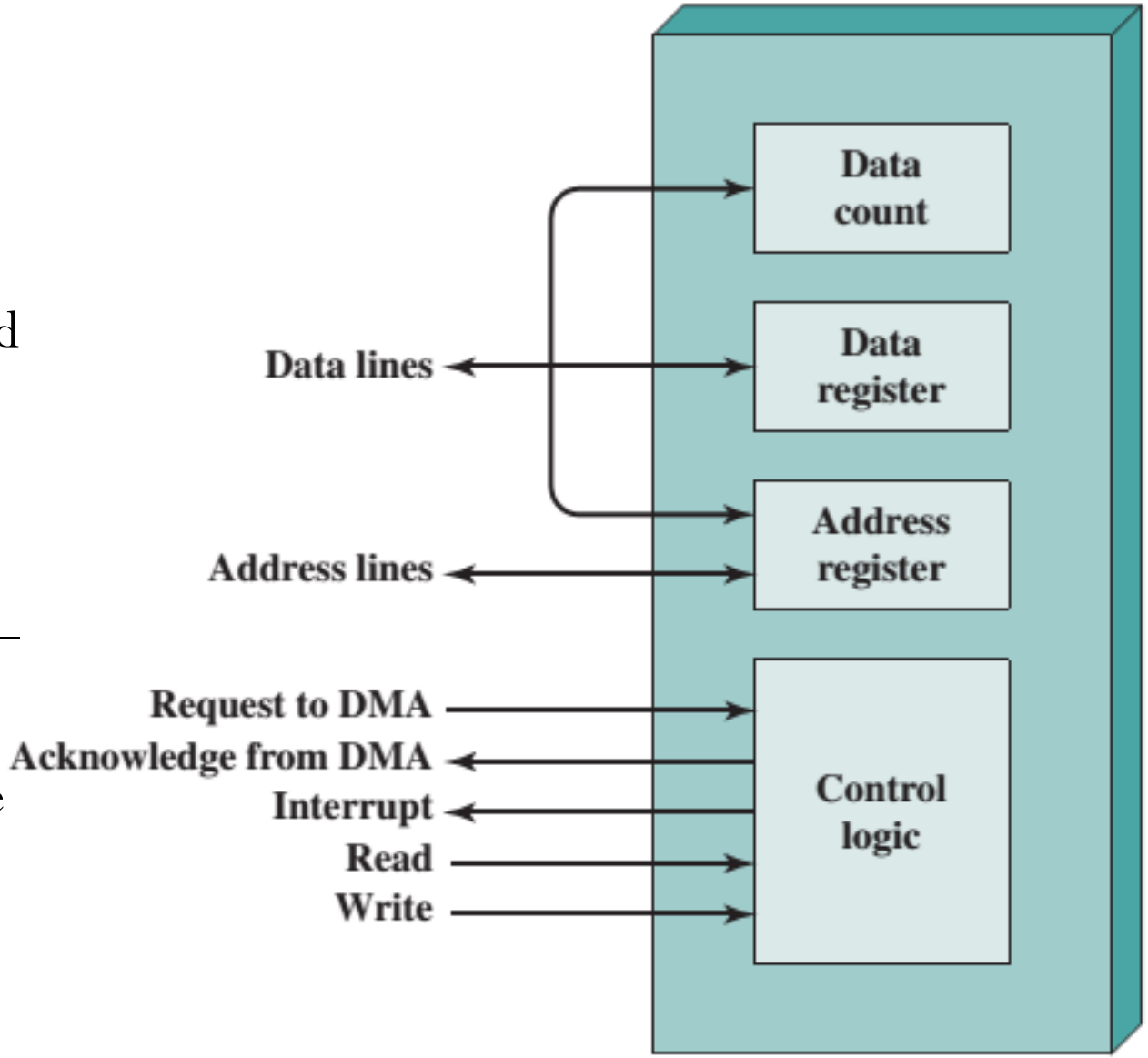
- **Bus arbitration**:
  - An I/O module device must first gain control of the bus before it can raise the interrupt request line
  - Only one module can raise the line at a time
  - When the processor detects the interrupt, it responds on the interrupt acknowledge line
  - The requesting module then places its vector on the data lines
- **Priority**
  - I/O module devices are often assigned a priority **to decide which module is to be serviced first**
    - With multiple lines, the interrupt line with the highest priority is chosen
    - With software polling, the order in which modules are polled determines the priority
    - The order of modules on a daisy chain determines their priority
    - In bus arbitration, highest priority module gets hold of the **IRL**

Direct Memory Access

- **Disadv. of Programmed I/O** and **Interrupt Driven I/O**
  - I/O transfer rate is limited by the speed with which the processor can test and service a device
  - Processor is actively involved in an I/O transfer
- **Direct Memory Access (DMA)** is more efficient for transfer of large volumes of data from the memory module directly to I/O device
- DMA module is an additional module – like I/O module
  - Shared bus connects DMA to all of the modules
  - DMA modules uses the system bus when it is not being used by the processor to directly transfer data from memory to I/O device

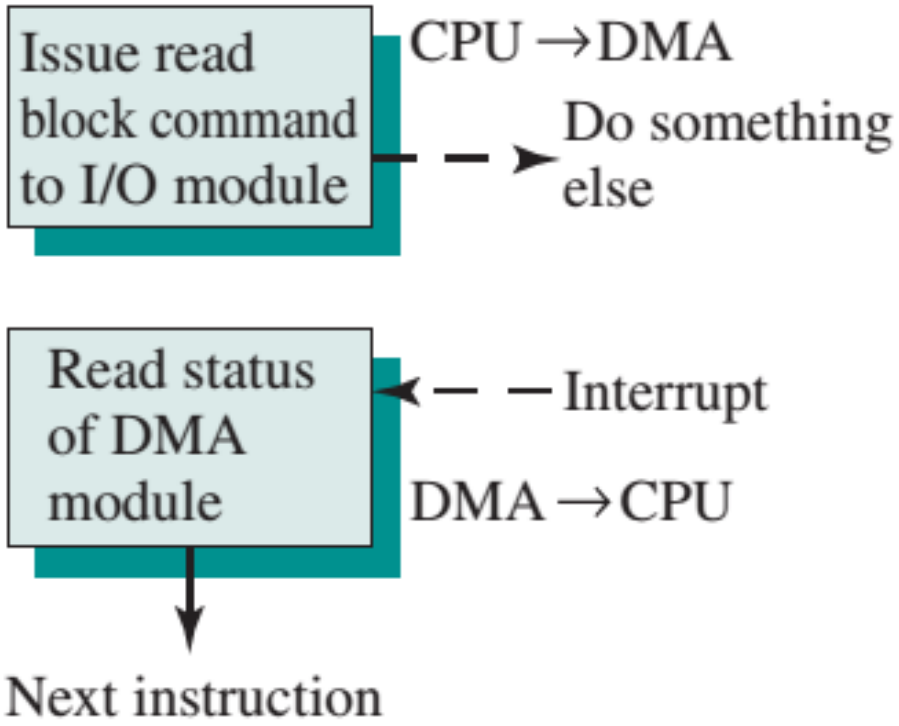
Direct Memory Access

- When the processor wishes to read or write a block of data, it issues a command to the DMA module (**or DMA controller**)
- Processor sends the following info to the DMA:
  - **Whether a read or write operation** – using the r/w control line between processor and DMA module
  - **Address of the I/O device** – using the data lines
  - **Starting location in memory** – using the data lines; DMA module stores it in address register
  - **Number of words involved** – using the data lines; DMA module stores it in data count register



Direct Memory Access

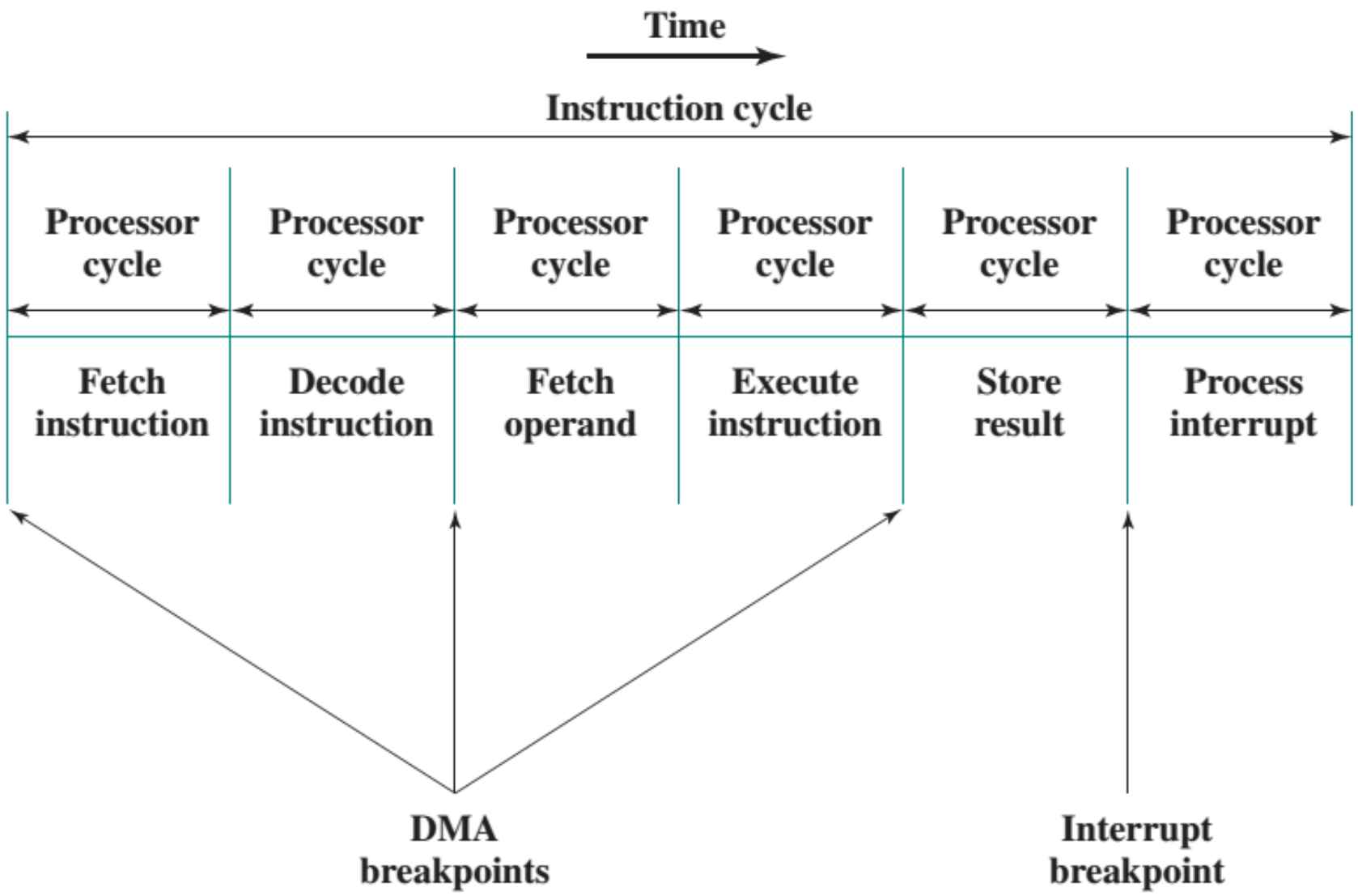
- Processor delegates the I/O operation and continues with other work
- DMA module transfers entire block of data without going through the processor
- When transfer is complete, the DMA module sends an interrupt signal to the processor
- Processor is involved only in the beginning and end of a transfer



Direct Memory Access

- DMA module needs to transfer data to and from memory over the **system bus**
- Two ways to use the system bus:
  - DMA module must use the bus only when the processor does not need it – for e.g., in the fetch-execute cycle during the execute phase
  - Force the processor to suspend operation temporarily – referred to as **cycle stealing**, because the DMA module in effect steals a bus cycle
- In cycle stealing, the processor temporarily halts – not to execute some other program, but to allow the DMA to use the bus

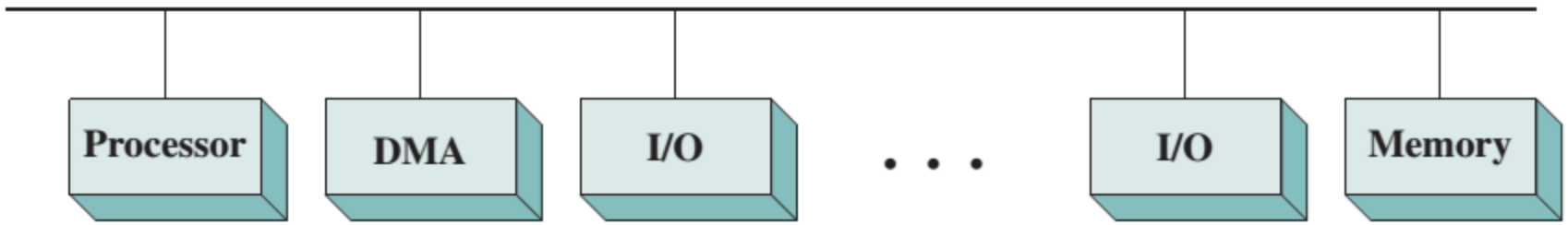
Direct Memory Access



Direct Memory Access

- Different types of DMA module configurations

1. **Single bus, detached DMA**
   - All modules share the same system bus
   - Similar to programmed I/O, exchanges data between memory and an I/O module through the DMA module
   - Inexpensive but inefficient – transfer of a word consumes two bus cycles.

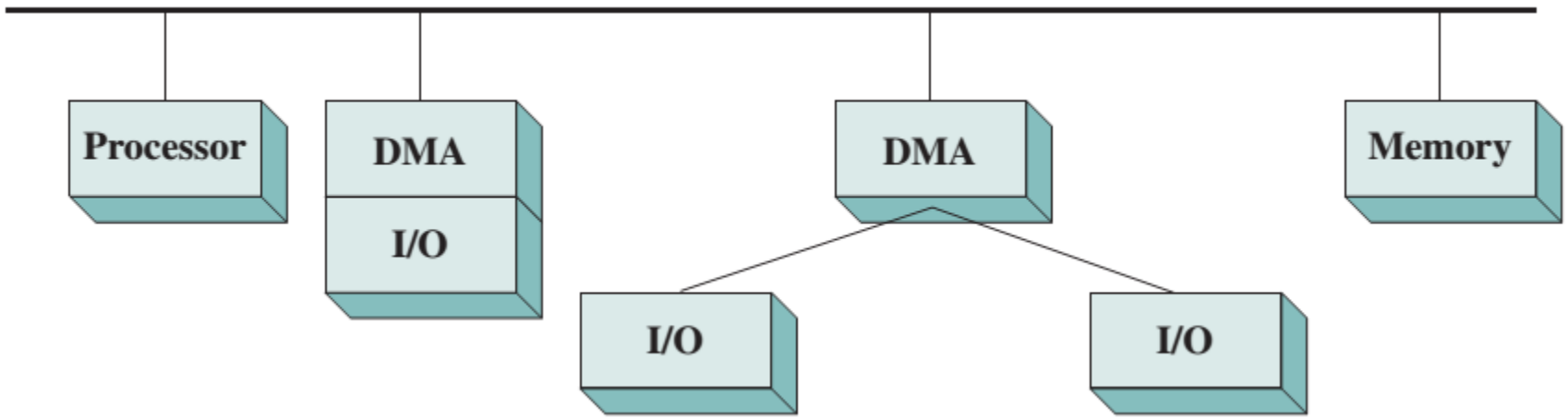


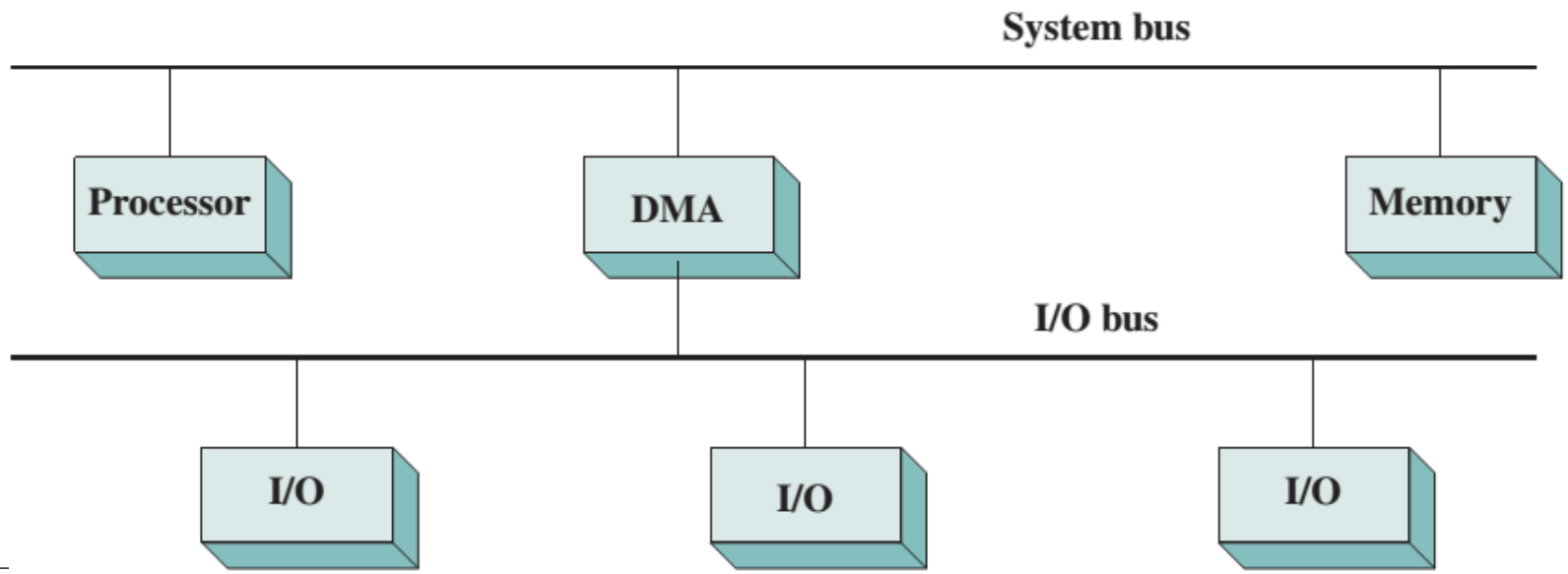
(a) Single-bus, detached DMA

Direct Memory Access

2. Single bus, integrated DMA module

- There is a path between the DMA module and one or more I/O modules that does not include the system bus
- Number of required bus cycles can be cut substantially
- DMA logic may actually be a part of an I/O module, or it may be a separate module that controls one or more I/O modules.



Direct Memory Access

3. **I/O bus**
   - I/O modules are connected to the DMA module using an I/O bus
   - Reduces the number of I/O interfaces in the DMA module to one; an easily expandable configuration
   - System bus is used by the DMA module only to exchange data with memory
   - The exchange of data between the DMA and I/O modules takes place in the system bus.